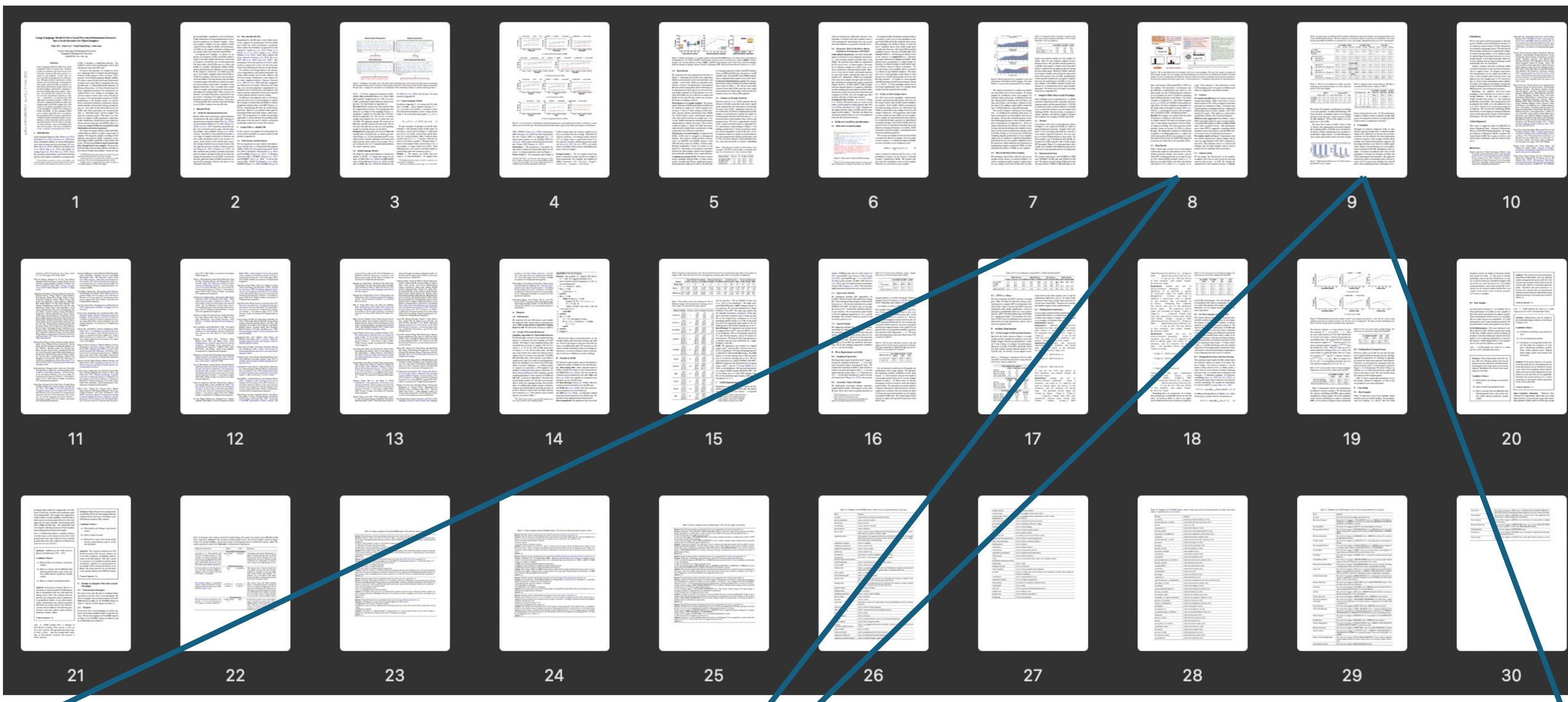

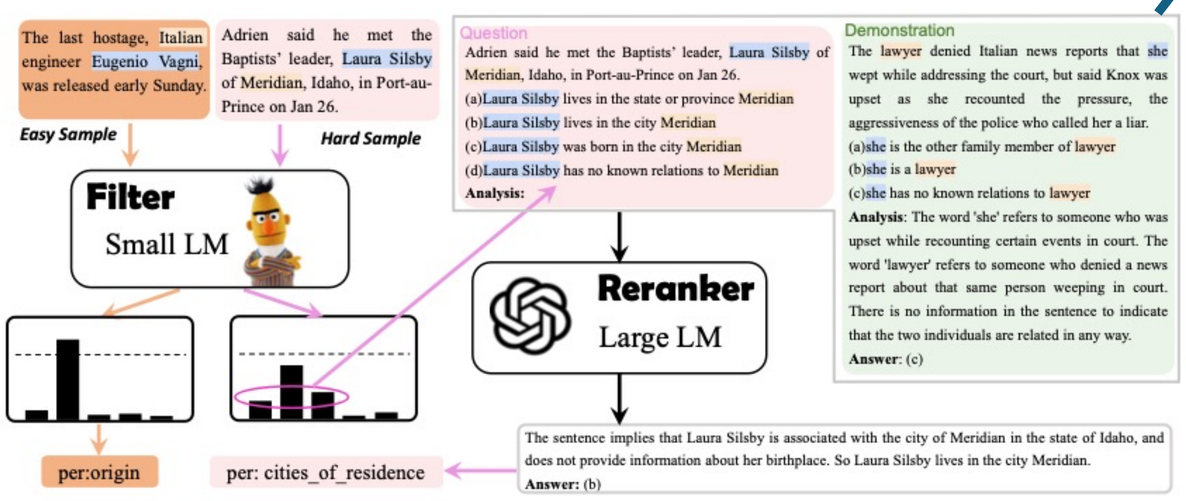

Figure 6: The overall architecture of our adaptive *filter-then-rerank* paradigm. We color easy samples in orange and hard samples in pink. For easy samples, the final predictions are exactly from the SLM-based methods. For hard samples, the top-N predictions from SLMs are fed into LLMs as the format of multiple-choice questions (pink box). The question is paired with demos (green box). LLMs rerank these N candidates and generate the final prediction.

filter, and Vicuna-13B, InstructGPT or GPT-4 as the reranker. The threshold τ to determine sample difficulty is optimized on the valid set. For hard sample, the top-3 SLM predictions and None (if not included) are feed to LLMs for reranking. Each LLM prompt has 4-shot demos. See demo examples in Appendix G.1. We follow templates in Lu et al. (2022a) for TACREV and carefully design others. See these templates in Appendix G.2. We adopt chain-of-thought reasoning (Wei et al., 2022b), *i.e.,* prefacing the answer with an explanation, to facilitate LLMs' reranking procedure.

Baseline We compare our method with two kinds of baselines to validate its effectiveness.

(1) LLMs with ICL: We follow the prompts in Section 3.3 and conduct experiments on three LLMs.

(2) Supervised SLMs: We follow previous SoTA methods shown in Section 3.4 (FSLS or KnowPrompt). We additionally combine two SLMs with ensemble or reranking approach (*i.e.,* replace the LLM with another SLM as the reranker) to verify that improvements from our SLM-LLM integrated system are not solely due to the ensemble effects.

5.3 Main Results

Table 3 shows that our *filter-then-rerank* method consistently improves performance across three datasets and nine settings. For instance, with InstructGPT, reranking provides an average F1 gain of 2.4% without SLM ensemble (Lines 4 vs. 7). Based on ensemble SLMs as the filter, our method still achieves 2.1% (Lines 5 vs. 8) gains on average. This confirms (1) the effectiveness of the LLM reranking and (2) its gains are different and (almost) orthogonal to the SLM ensemble.

5.4 Analysis

Few makes big difference Our method selectively reranks hard samples. Table 4 shows that (1) only a minor fraction (0.5%~10%) of samples are deemed hard and are reranked by LLMs. (2) Despite their limited quantity, reranking results in a substantial performance boost on these samples (10%~25% absolute F1 gains). This uplift on a small subset significantly enhances the overall performance.

GPT-4 is more aggressive From Tables 3 and 4, GPT-4 generally improves more on hard samples, yet InstructGPT surpasses GPT-4 in NER and RE tasks when evaluated overall. This discrepancy arises from GPT-4's aggressive reranking which introduces more true positives. InstructGPT, however, focuses more on reducing false positives.

Few makes small cost Figure 7 demonstrates that our method impressively reduces budget and latency by approximately 80%~90% compared to direct ICL. This reduction is due to (1) fewer LLM callings (only for hard samples) and (2) shorter prompts (fewer candidate labels and demos).

5.5 Ablation Study

We investigate the effectiveness of the modules in adaptive *filter-then-rerank* system by removing each of them in turn: (1) **CoT**: We exclude the explantion for each examples in demo. (2) **Demo**: We remove all examples, rendering the reranking a zero-shot problem. (3) **LF** (label filtering): We retain all labels as candidate choices for reranking, instead of only the top-N labels from the SLMs. (4) **AD** (adaptive): We feed all samples, not just hard ones, to the LLMs.

We show their results in Table 5 and see that (1) Demos with explanations consistently enhance the reranking ability of LLMs across all datasets. (2) Demos without explanations also contribute to performance improvement. (3) Label filtering results in gains and notably reduces the demo length, hence cutting inference costs. (4) The performance collapses without a filter to identify sample difficulty, reiterating the need for an integrated SLM-LLM system to complement each other.

Table 3: Overall results of LLM-based ICL methods, SLM-based supervised methods, and our proposed *filter-then-rerank* (SLM+LLM) methods. The best results are in bold face and the second best are underlined. All results except InstructGPT and GPT-4 are averaged over 5 runs, and sample standard deviations are in the round bracket.

		FewNERD (NER)			TACREV (RE)			ACE (ED)		
		5-shot	10-shot	20-shot	20-shot	50-shot	100-shot	5-shot	10-shot	20-shot
LLM	CODEX	53.8(0.5)	54.0(1.4)	55.9(0.5)	59.1(1.4)	60.3(2.4)	62.4(2.6)	47.1(1.2)	47.7(2.8)	47.9(0.5)
	InstructGPT	53.6(–)	54.6(–)	57.2(–)	60.1(–)	58.3(–)	62.7(–)	52.9(–)	52.1(–)	49.3(–)
	GPT-4	-	-	57.8(–)	-	-	59.3(–)	-	-	52.1(–)
SLM	Previous SoTA	59.4(1.5)	61.4(0.8)	61.9(1.2)	62.4(3.8)	68.5(1.6)	72.6(1.5)	55.1(4.6)	63.9(0.8)	65.8(2.0)
	+ Ensemble (S)	59.6(1.7)	61.8(1.2)	62.6(1.0)	64.9(1.5)	71.9(2.2)	74.1(1.7)	56.9(4.7)	64.2(2.1)	66.5(1.7)
	+ Rerank (S)	59.4(1.5)	61.0(1.7)	61.5(1.7)	64.2(2.3)	70.8(2.3)	74.3(2.2)	56.1(0.3)	64.0(1.0)	66.7(1.7)
SLM + LLM	**Vicuna-13B**									
	+ Rerank (L)	60.0(1.8)	61.9(2.1)	62.2(1.4)	65.2(1.4)	70.8(1.6)	73.8(1.7)	56.9(4.0)	63.5(2.7)	66.0(2.6)
	+ Ensemble (S) + Rerank (L)	59.9(0.7)	62.1(0.7)	62.8(1.1)	66.5(0.5)	73.6(1.4)	75.0(1.5)	57.9(5.2)	64.4(1.2)	66.2(2.4)
	InstructGPT									
	+ Rerank (L)	60.6(2.1)	62.7(0.8)	63.3(0.6)	66.8(2.6)	72.3(1.4)	75.4(1.5)	57.8(4.6)	65.3(1.7)	67.3(2.2)
	+ Ensemble (S) + Rerank (L)	**61.3**(1.9)	**63.2**(0.9)	**63.7**(1.8)	**68.9**(1.3)	**74.8**(1.3)	**76.8**(1.2)	59.5(3.7)	65.3(1.9)	67.8(2.1)
	GPT-4									
	+ Rerank (L)	60.8(2.3)	62.6 (2.7)	63.0(1.3)	65.9(2.7)	72.3(0.3)	74.5(1.5)	59.6(2.9)	64.9(2.5)	67.1(2.5)
	+ Ensemble (S) + Rerank (L)	61.1(2.2)	62.8(0.9)	63.6(1.2)	68.6(1.3)	73.9(1.4)	75.9(2.4)	**60.9**(3.9)	**65.6**(1.5)	**67.8**(1.7)

Table 4: The F1-score differences before and after reranking on the reranked samples, as well as their proportion of the total samples.

	GPT-4				InstructGPT			
	before	after	△	ratio	before	after	△	ratio
FewNER	31.9	40.7	8.8	3.2%	31.4	28.3	−3.1	3.3%
TACREV	25.3	43.0	17.7	9.1%	33.8	43.4	9.6	7.1%
ACE05	31.1	57.9	26.8	1.6%	35.6	55.7	20.1	0.5%

Table 5: Ablation study on three datasets. The filter is ensembled SLMs and the reranker is GPT-4.

CoT	Demo	LF	AD	FewNERD (20-shot)	TACREV (100-shot)	ACE05 (20-shot)
✓	✓	✓	✓	**63.6**(1.2)	**75.9**(2.4)	**67.8**(1.7)
✗	✓	✓	✓	63.2(1.2)	75.4(2.4)	67.2(1.7)
✗	✗	✓	✓	63.0(1.4)	74.9(2.2)	66.6(1.5)
✗	✗	✗	✓	62.4(2.1)	73.8(2.5)	66.5(1.3)
✗	✗	✗	✗	12.5(2.7)	59.9(6.0)	5.4(1.1)
Previous SoTA methods				62.6(1.0)	74.1(1.7)	66.5(1.7)

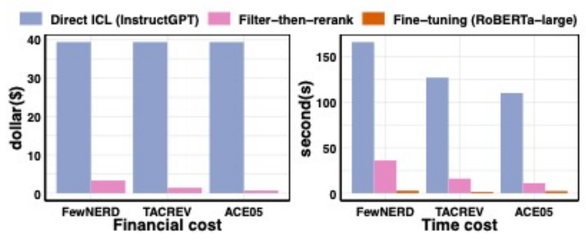

Figure 7: The financial and time cost over 500 sentences. InstructGPT as the reranker.

6 Conclusion

Through an extensive empirical study on nine datasets spanning four IE tasks, we find that LLMs, despite their superiority in extreme low-resource scenarios, are not effective few-shot information extractors in general. They struggle with IE-related prompts, have limited demonstration capacity, and incur high inference costs. However, LLMs significantly improve the performance on *hard* samples when combined with SLM. Building on these insights, we propose an adaptive *filter-then-rerank* paradigm to leverage the strengths of SLMs and LLMs and mitigate their limitations. This approach consistently achieves promising results, with an average 2.4% F1 gain across multiple few-shot IE tasks, while minimizing latency and budget costs.

Question: If I drop the green module shown in Figure 6, what is the absolute percentage drop on 100-shot TACREV setting?

Response: The absolute percentage drop on the 100-shot TACREV setting when the green module is dropped can be found in Table 4. For the 100-shot TACREV setting, the F1 score before reranking is 72.0, and after reranking, it is 74.3. The absolute percentage drop is calculated as follows:

Absolute Percentage Drop = ((F1 score before reranking - F1 score after reranking) / F1 score after reranking) × 100

Absolute Percentage Drop = ((74.3 - 72.0) / 74.3) × 100

Absolute Percentage Drop = (2.3 / 74.3) × 100

Absolute Percentage Drop ≈ 3.1%

So, the absolute percentage drop on the 100-shot TACREV setting when the green module is dropped is approximately 3.1%.

Error Type: Reasoning Error

Analysis: The formula listed in model response is the relative percentage drop, instead of the absolute percentage drop.